

Auto Scaling Group Implementation with Load Balancer

Objective

To implement **Auto Scaling** for automatically managing EC2 instances based on traffic demand, and to integrate it with a **Load Balancer** to ensure high availability, fault tolerance, and scalability.

Step 1: Prepare the Network Environment

Concept:

Auto Scaling and Load Balancing require a stable networking foundation.

Implementation:

- Use an existing **VPC**
- Select **multiple public subnets** across different Availability Zones
- Ensure **Internet Gateway** and route tables are properly configured

This setup allows the Auto Scaling Group to launch instances in multiple AZs.

Step 2: Create Security Groups

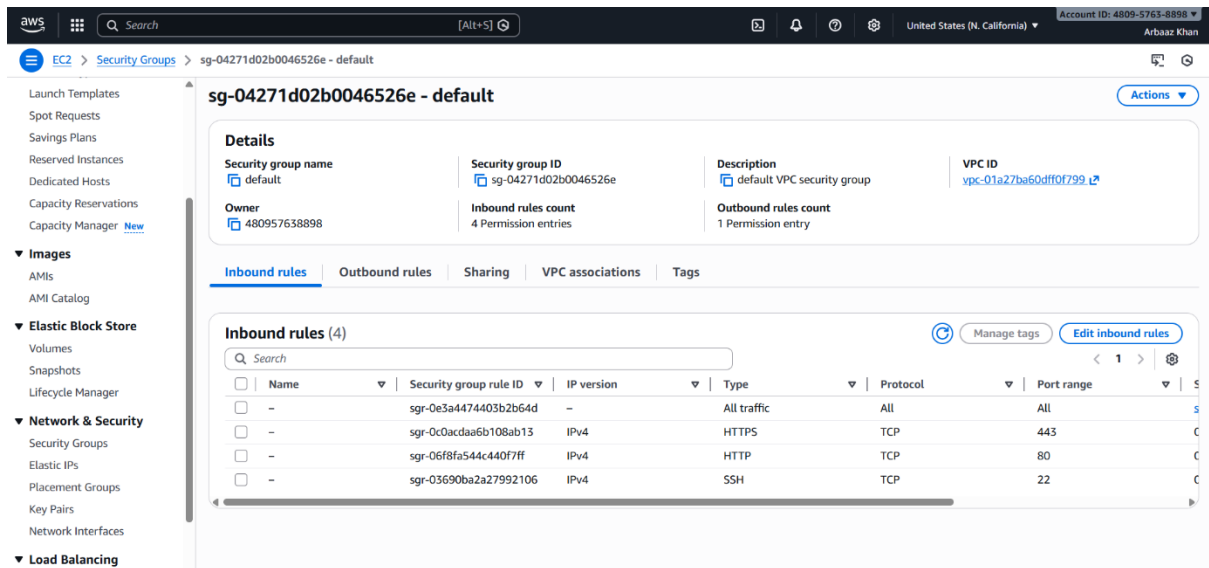
Concept:

Security groups control inbound and outbound traffic.

Implementation:

- **Load Balancer Security Group**
 - Allow inbound HTTP (port 80) from the internet
- **EC2 Instance Security Group**
 - Allow inbound HTTP (port 80) only from the Load Balancer security group

This ensures backend instances are not directly exposed.



Step 3: Create Launch Template – Web Template 1

Concept:

A Launch Template defines instance configuration used by Auto Scaling.

Configuration Included:

- Ubuntu AMI
- Instance type (Free Tier eligible)
- Key pair
- EC2 security group
- User Data script

User Data Script Purpose:

- Install Nginx
- Deploy sample webpage
- Identify instance uniquely

Step 4: Create Launch Template – Web Template 2

Concept:

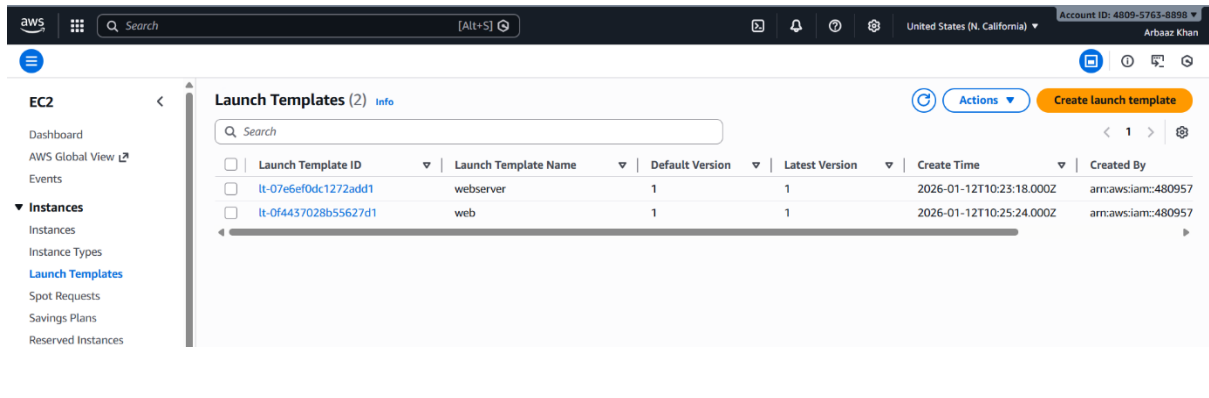
Using multiple launch templates allows flexible deployments (different versions, configurations, or environments).

Configuration:

- Same base settings as Template 1
- Different HTML message or version identifier

- Used to demonstrate instance variation within Auto Scaling

Both templates represent **scalable, reusable instance blueprints**.



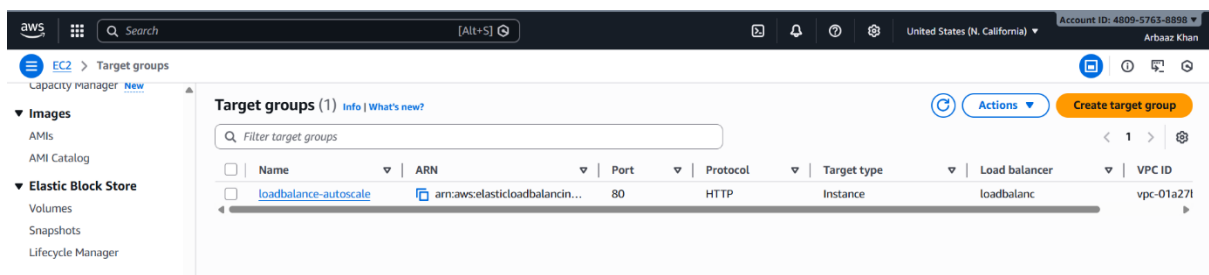
Step 5: Create Target Group

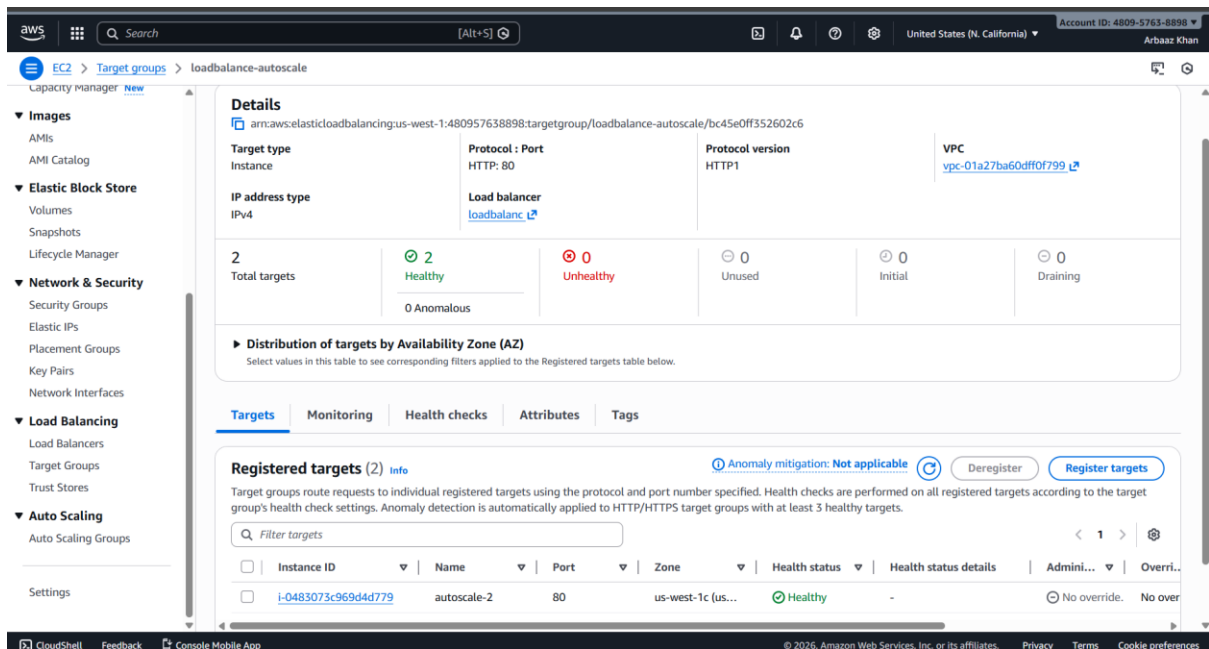
Concept:

Target Groups define where the Load Balancer sends traffic.

Implementation:

- Protocol: HTTP
- Port: 80
- Target type: Instances
- Health checks enabled
- No manual instance registration (ASG handles this)





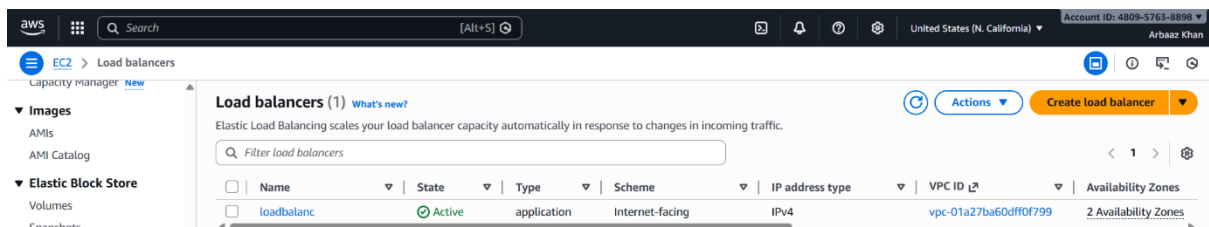
Step 6: Create Application Load Balancer (ALB)

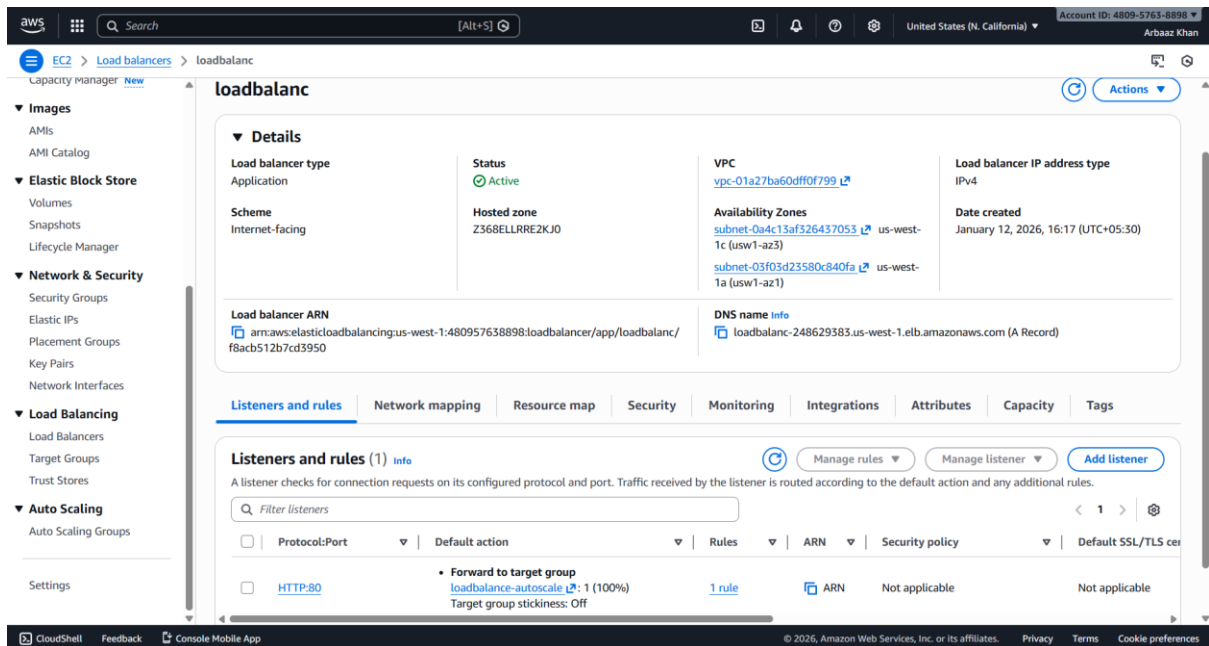
Concept:

ALB distributes incoming traffic across healthy EC2 instances.

Implementation:

- Internet-facing ALB
- Attach public subnets (Multi-AZ)
- Assign Load Balancer security group
- Associate with the target group





Step 7: Create Auto Scaling Group (ASG)

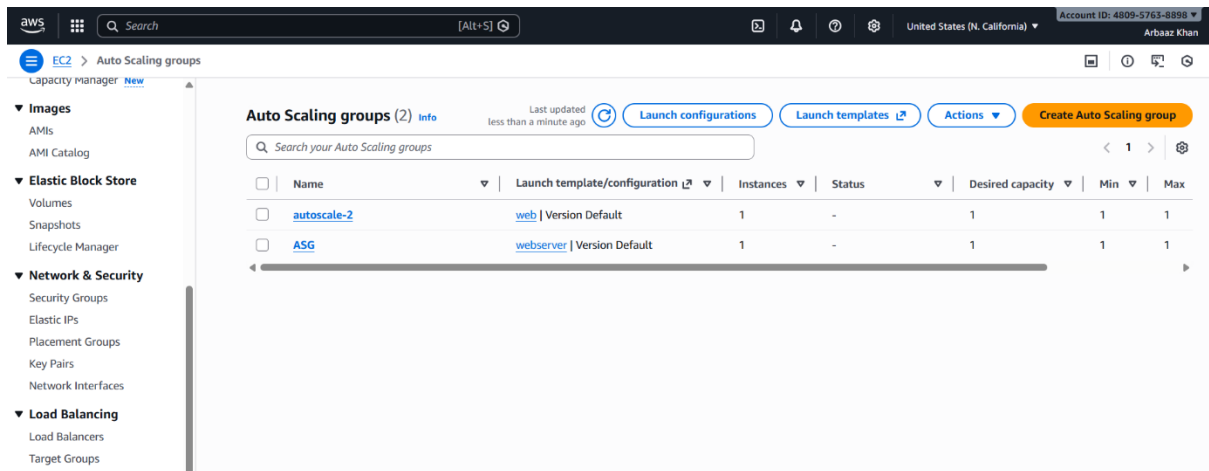
Concept:

Auto Scaling Groups automatically launch and terminate instances based on demand.

Implementation:

- Choose **Mixed Instance Policy**
- Attach **Launch Template 1** and **Launch Template 2**
- Select multiple Availability Zones
- Set capacity:
 - Minimum: 1
 - Desired: 2
 - Maximum: 4

This allows scaling across templates and AZs.



Step 8: Attach Load Balancer to ASG

Concept:

Attaching ASG to ALB ensures new instances automatically receive traffic.

Implementation:

- Attach target group to Auto Scaling Group
- Health checks handled by ALB
- Unhealthy instances replaced automatically

Step 9: Configure Scaling Policies

Concept:

Scaling policies control when instances are added or removed.

Implementation:

- Target tracking scaling
- Metric: Average CPU Utilization
- Scale out at high CPU usage
- Scale in when demand reduces

This ensures **cost-efficient elasticity**.

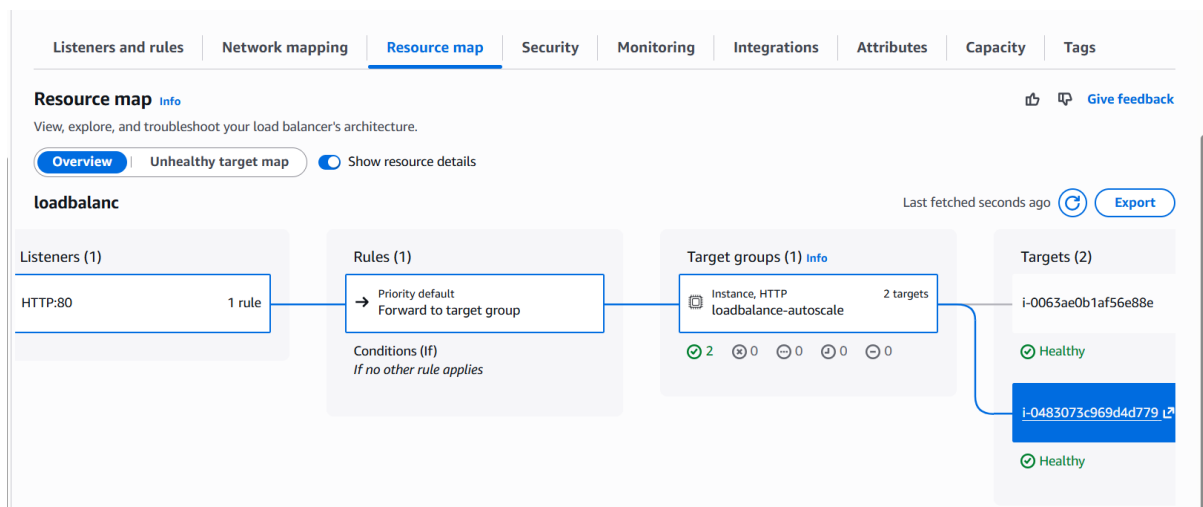
Step 10: Launch and Validate Auto Scaling

Concept:

Validation confirms correct scaling behavior.

Testing:

- Access Load Balancer DNS
- Refresh page to see different instance responses
- Generate CPU load to trigger scale-out
- Stop an instance to observe automatic replacement



The screenshot shows the AWS Management Console for EC2 instances. The table lists the following instances:

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4
autoscaling	i-0063ae0b1af56e88e	Terminated	m1.small	-	View alarms +	us-west-1c	-
autoscaling	i-0f8eac91c713799a1	Running	m1.small	Initializing	View alarms +	us-west-1c	ec2-54-241-
autoscale-2	i-0483073c969d4d779	Running	m1.small	2/2 checks passed	View alarms +	us-west-1c	ec2-54-193-
instance-02	i-0ee75c9bcc7148add	Terminated	t3.micro	-	View alarms +	us-west-1a	-
instance-03	i-0a455874f6b88062f	Terminated	t3.micro	-	View alarms +	us-west-1a	-
instance-01	i-0ce96741d74cc97ef	Terminated	t3.micro	-	View alarms +	us-west-1a	-

Server IP: 172.31.5.43

Server IP: 172.31.15.232

End-to-End Traffic Flow

1. User sends HTTP request
 2. Request reaches Application Load Balancer
 3. ALB forwards traffic to target group
 4. Auto Scaling Group provides healthy EC2 instances
 5. Nginx serves response
 6. Auto Scaling replaces failed instances automatically
-

Outcome

- Load-balanced, auto-scaling web infrastructure
- High availability across Availability Zones
- Automated recovery from failures
- Zero-downtime scaling
- Production-ready AWS architecture